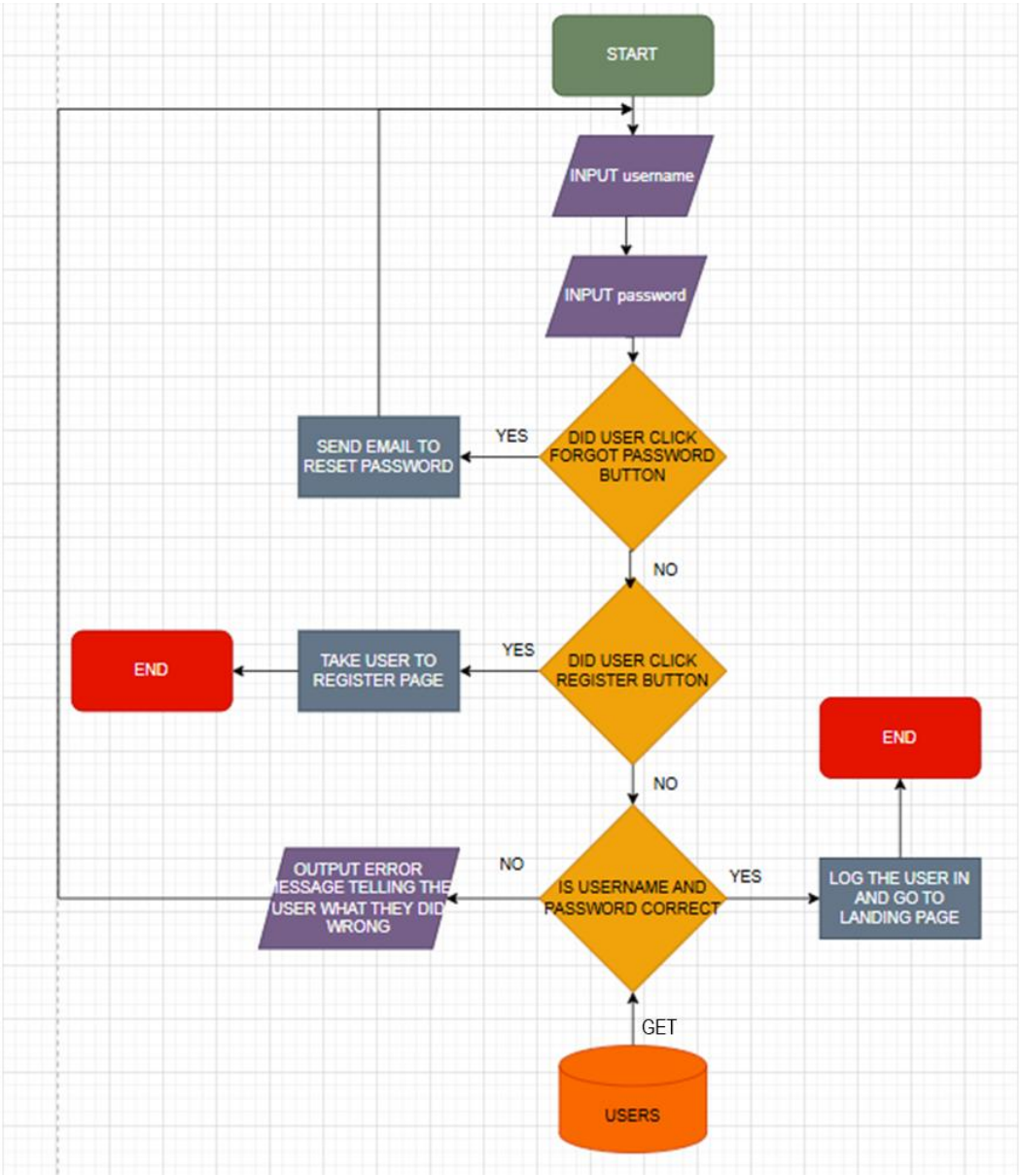
Contents

Flow Charts 2
Log In Function 2
Register Function 3
Add Item to basket 4
Create an Order 5
Updating Products 6
Database Design 7
Data Dictionaries 9
AspNetUsers 9
AspNetRoles 9
AspNetUserRoles 9
AspNetUserClaims 10
AspNetRoleClaims 10
AspNetUserLogins 10
AspNetUserTokens 10
Producers 10
Products 11
Orders 11
OrderProducts 12
Basket 12
BasketProducts 12
Stamps 12
ProductStamps 13
DiscountCodes 13
Site map 13
Test Strategy 13
Visual Design 25
Styles 25
Dark Mode 25
Background 25
Font 25
Interactive 26
Light Mode 26
Background 26

Font 26
Interactive 26
Buttons/Cards 26
Accent Colours 26

Flow Charts

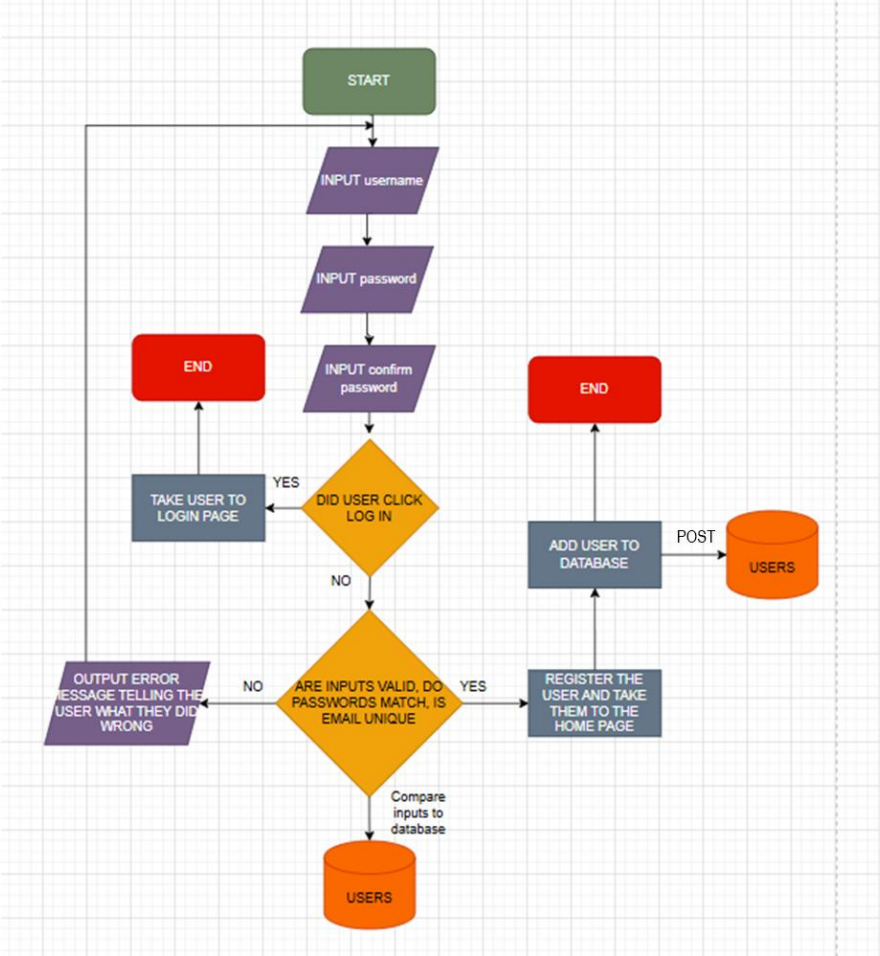
Log In Function



The user will be prompted to enter their username and password the system will then check if the user clicked other options like forgot password or register if they did it will take the appropriate actions like sending a reset email link or taking the user to the register page if not

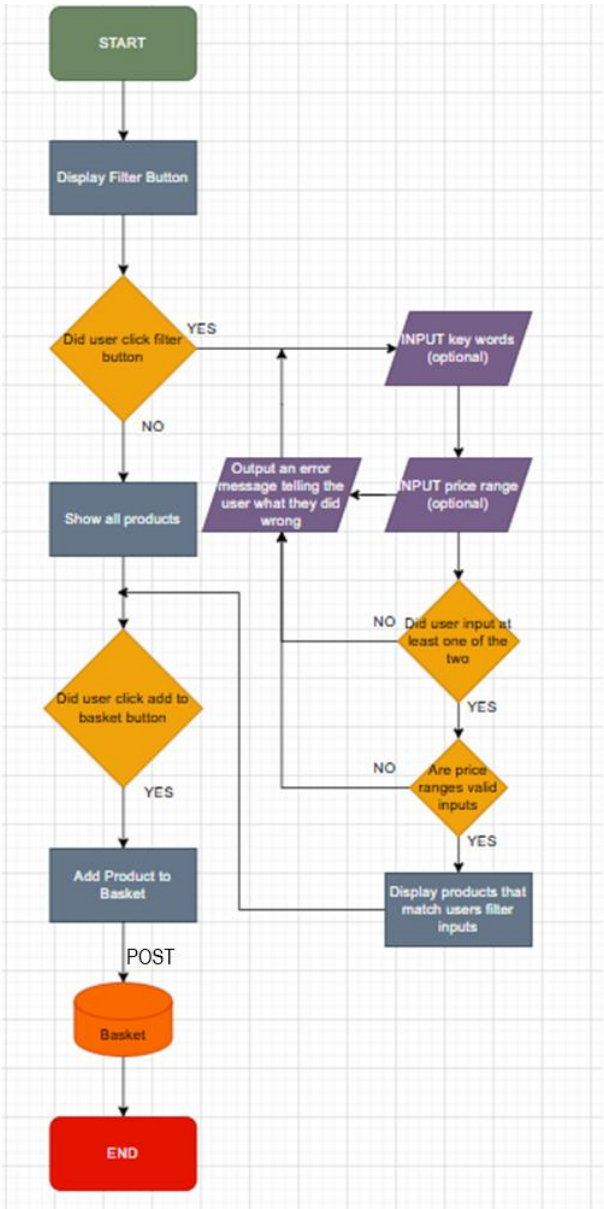
the system will check if the username and password match the ones in the database and if not, it will send an error message telling the user what they did wrong if its correct the system will log the user in return them to the landing page.

Register Function



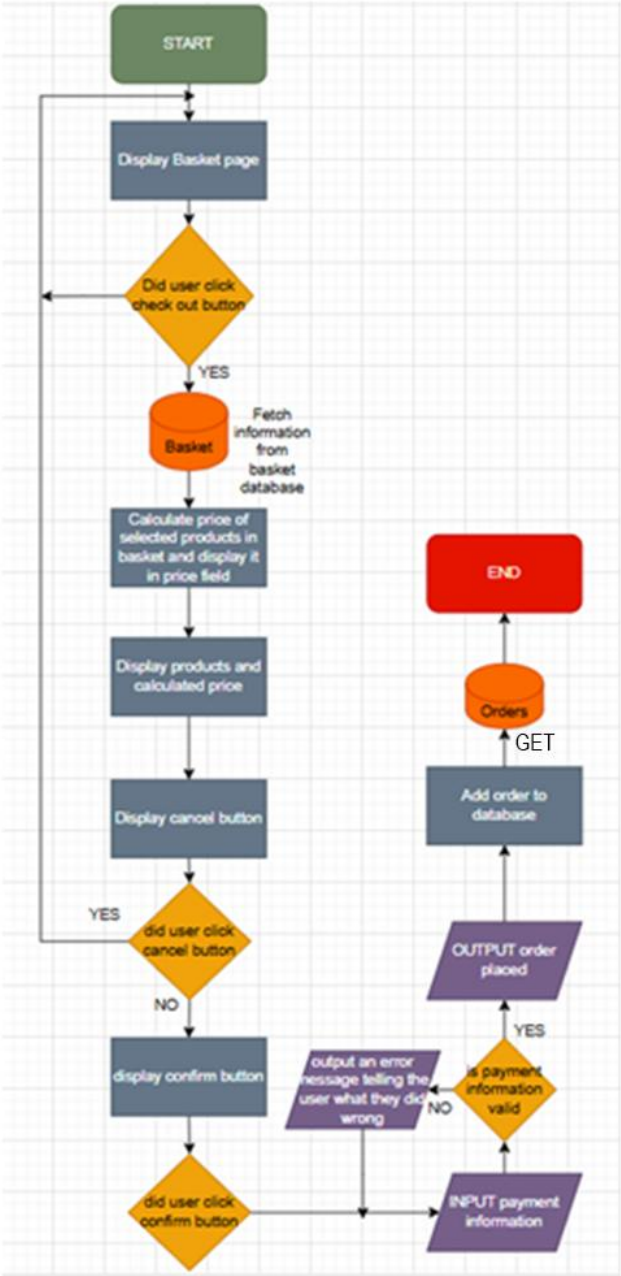
The user will be prompted to enter their username and password and password again to confirm it, the system then checks if the two passwords match and if the email is unique to any other account from the database, then they can proceed to add the user to the database if everything is good but if there are any issues the system prompts the user with an error message explaining what they have done wrong and asks them to try again.

Add Item to basket



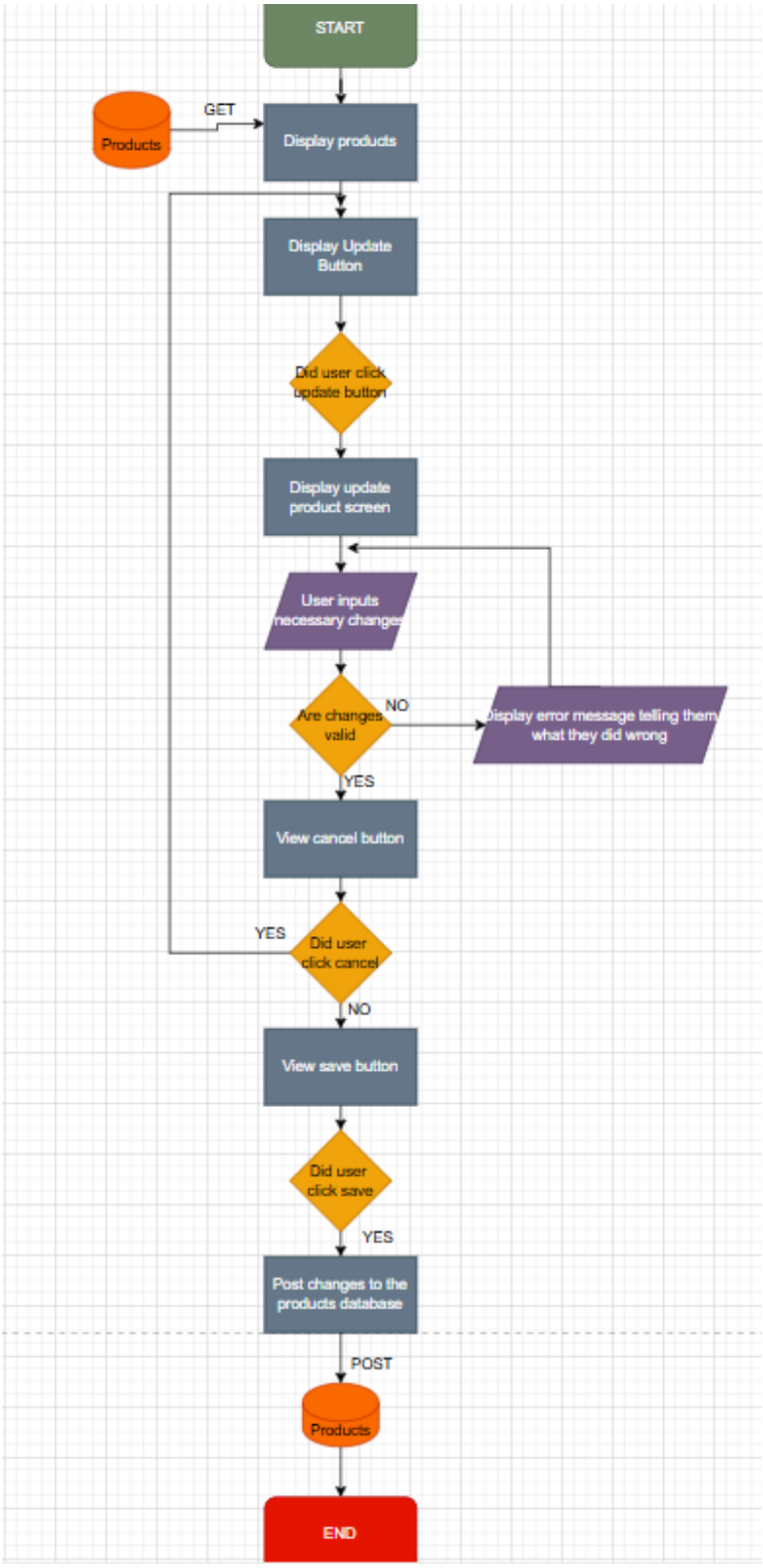
The user being on the main shopping page with all products will be given the option to either browse the full length on products or filter to give more specific results, if the user manages to enter invalid characters or data types to the filter fields, they are prompted on what they have done wrong and are allowed to try again. Once they have found the results they want or if they didn't filter at all, they can then click the "add to basket button" located below the item and this will prompt the web app to add that specific item to the user's basket.

Create an Order



From the basket page the user is given the option to checkout their selected items, if the user click the checkout button the application displays the products and calculates the price and then they can either cancel or confirm their order, if they cancel their order they are redirected to the basket page and if they confirm the order they are prompted to enter their payment details, the web app checks if the payment details are valid or not, if not they are told what they have done wrong and are asked to try again and If they are right and the payment goes through then the order is accepted and posted to the database for further actions.

Updating Products



When a producer needs to make an update to their stock or products they have listed from the producer dashboard page they can see their products they've listed, there they can click the update button next to the product, this button redirects them to the edit page for the selected product and there they can make the necessary changes, the application then checks if the changes they've made are valid, and if not it prompts them to try again and if the changes are valid it posts the changes to the database making them permanent till updated again.

Database Design

AspNetRoles
Id string
Name string
NormalizedName string
ConcurrencyStamp string
AspNetRoleClaims
Id string
RoleId string
ClaimType string
ClaimValue string
AspNetUsers
Id string
UserName string
NormalizedUserName string
Email string
NormalizedEmail string
EmailConfirmed boolean
PasswordHash string
SecurityStamp string
ConcurrencyStamp string
PhoneNumber string
PhoneNumberConfirmed boolean
TwoFactorEnabled boolean
LockoutEnd datetime
LockoutEnabled boolean
AccessFailedCount integer
AspNetUserRoles
Id string
UserId string
RoleId string
AspNetUserClaims
Id string
UserId string
ClaimType string
ClaimValue string
AspNetUserLogins
LoginProvider string
ProviderKey string
ProviderDisplayName string
UserId string
Stamps
StampsId integer
StampName string
StampDescription string
AspNetUserTokens
UserId string
LoginProvider string
Name string
Value string
Basket
BasketId integer
UserId string
Producers
ProducersId integer
UserId string
BusinessName string
BusinessDescription string
BusinessBasedIn string
BasketProducts
BasketProductsId integer
BasketId integer
ProductsId integer
Quantity integer
Orders
OrdersId integer
UserId string
CreatedAt datetime
IsDelivery boolean
ScheduledTime datetime
DeliveryAddress string
DeliveryFee float
TotalPrice float
UsedDiscount boolean
DiscountName string
ProductStamps
ProductStampsId integer
ProductsId integer
StampsId integer
Products
ProductsId integer
ProducersId integer
ProductName string
Price float
Stock integer
Description string
IsAvailable boolean
ImageUrl string
DiscountCodes
DiscountCodeId integer
Code string
Percentage float
IsActive boolean
OrderProducts
OrderProductsId integer
OrdersId integer
ProductsId integer
Quantity integer
dbdiagram.io

Data Dictionaries

AspNetUsers

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
ID	String	Yes			Primary
USERNAME	String	Yes			None
NORMALIZEDUSERNAME	String	Yes			None
EMAIL	String	Yes		Must include @, valid email structure, domain name	None
NORMALIZEDEMAIL	String	Yes			None
EMAILCONFIRMED	Bool	Yes		Users must confirm their email	None
PASSWORDHASH	String	Yes			None
SECURITYSTAMP	String	No			None
CONCURRENCYSTAMP	String	No			None
PHONENUMBER	String	No			None
PHONENUMBERCONFIRMED	Bool	No			None
TWOFACTORENABLED	Bool	No			None
LOCKOUTEND	DateTime	No			None
LOCKOUTENABLED	Bool	No			None
ACCESSFAILEDCOUNT	Int	No			None

AspNetRoles

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
ID	String	Yes			Primary
NAME	String	Yes			None
NORMALIZEDNAME	String	Yes			None
CONCURRENCYSTAMP	String	Yes			None

AspNetUserRoles

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
ID	Int	Yes			Primary
USERID	String	Yes	Users Id		Foreign
ROLEID	String	Yes	Roles Id		Foreign

AspNetUserClaims

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
ID	Int	Yes			Primary
USERID	String	Yes	Users Id		Foreign
CLAIMTYPE	String	No			None
CLAIMVALUE	String	No			None

AspNetRoleClaims

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
ID	Int	Yes			Primary
ROLEID	String	Yes	Roles Id		Foreign
CLAIMTYPE	String	No			None
CLAIMVALUE	String	No			None

AspNetUserLogins

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
LOGINPROVIDER	String	Yes			Primary
PROVIDERKEY	String	Yes			None
PROVIDERDISPLAYNAME	String	No			None
USERID	String	Yes	Users Id		Foreign

AspNetUserTokens

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
USERID	String	Yes	Users Id		Foreign
LOGINPROVIDER	String	Yes			None
NAME	String	Yes			None
VALUE	String	No			None

Producers

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
PRODUCERSID	Int	Yes			Primary
USERID	String	Yes	Users Id		Foreign
BUSINESSNAME	String	Yes			None
BUSINESSDESCRIPTION	String	No			None
BUSINESSBASEDIN	String	Yes			None

Products

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
PRODUCTSID	Int	Yes			Primary
PRODUCERSID	Int	Yes	Producers Id		Foreign
PRODUCTNAME	String	Yes			None
PRICE	Float	Yes		Must be greater than 0	None
STOCK	Int	Yes	0	Must be 0 or greater	None
DESCRIPTION	String	No			None
ISAVAILABLE	Bool	Yes	false		None
IMAGEURL	String	No			None
COLUMN	Data Type	Required	Default	Validation	None

Orders

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
ORDERSID	Int	Yes			Primary
USERID	String	Yes	Users Id		Foreign
CREATEDAT	DateTime	Yes	Current timestamp		None
ISDELIVERY	Bool	Yes			None
SCHEDULEDTIME	DateTime	No		Must be a future date	None
DELIVERYADDRESS	String	No		Required if IsDelivery is true	None
DELIVERYFEE	Float	No	0	Required if IsDelivery is true, must be 0 or greater	None
TOTALPRICE	Float	Yes		Must be greater than 0	None
USEDDISCOUNT	Bool	Yes	false		None
DISCOUNTNAME	String	No		Required if UsedDiscount is true	None

OrderProducts

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
ORDERPRODUCTSID	Int	Yes			Primary
ORDERSID	Int	Yes	Orders Id		Foreign
PRODUCTSID	Int	Yes	Products Id		Foreign
QUANTITY	Int	Yes		Must be greater than 0	None

Basket

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
BASKETID	Int	Yes			Primary
USERID	String	Yes	Users Id		Foreign

BasketProducts

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
BASKETPRODUCTSID	Int	Yes			Primary
BASKETID	Int	Yes	Basket Id		Foreign
PRODUCTSID	Int	Yes	Products Id		Foreign
QUANTITY	Int	Yes		Must be greater than 0	None

Stamps

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
STAMPSID	Int	Yes			Primary
STAMPNAME	String	Yes			None
STAMPDESCRIPTION	String	No			None

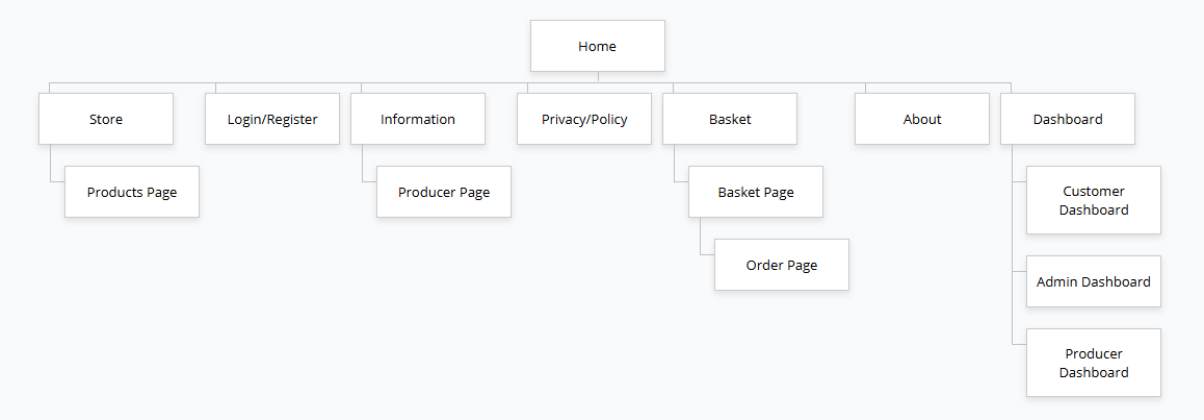
ProductStamps

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
PRODUCTSTAMPSID	Int	Yes			Primary
PRODUCTSID	Int	Yes	Products Id		Foreign
STAMPSID	Int	Yes	Stamps Id		Foreign

DiscountCodes

COLUMN	DATA TYPE	REQUIRED	DEFAULT	VALIDATION	KEY TYPE
DISCOUNTCODEID	Int	Yes			Primary
CODE	String	Yes		Must be unique	None
PERCENTAGE	Float	Yes		Must be between 1 and 100	None
ISACTIVE	Bool	Yes	true		None

Site map



Test Strategy

Date of Test	*Component to be Tested*	*Type of Test to be carried out*	*Prerequisites and dependencies*
14/04/26	Register	White box testing, black box testing, functionality testing, Security testing, Integration Testing.	The register function should be fully functional and implemented. Migrations and update database must be applied. The controller logic should be completed to handle user registration and error

			handling for incorrect inputs duplicate accounts and password complexity must be set up. Tests should include valid/invalid Test data needed. The log-in page must be created, and the migration and update database must be completed. There must be working controller logic. Testers must test valid/invalid credentials, session management, vulnerabilities (SQL injection) and password encryption. Example of test data: - Normal: Email: “Test@testemail.com,” Password: "Password123 ?”, Confirm Password: “Password123 ?” - Erroneous: Email: “falseemail” (Should contain a “@ symbol”, Should contain a domain e.g. “.com”), Password: “111” (Should be longer, should be a string, should have a capital letter and a symbol), Confirm Password: “1er” (Does not match original password) - Extreme: test@testemail.com, Password: “QwE12?” (Exactly 6 characters which is the minimum and includes 1 symbol which is the minimum)
14/04/26	Log in	White box testing, black box testing, functionality testing, Security testing, Integration Testing	Test data needed. The log-in page must be created, and the migration and update database must be completed. There must be working controller logic. Testers must test valid/invalid credentials, session management, vulnerabilities (SQL injection) and password encryption. Example of test data: - Normal: Email: “Test@testemail.com”, Password: “Password123 ?” - Erroneous: Email: “falseemail” (Should contain a “@ symbol”, Should contain a domain e.g. “.com”), Password: “pass” (Should be longer, should

			be a string, should have a capital letter and a symbol) - Extreme: “test@testemail.com”, Password: QwE12? (Exactly 6 characters which is the minimum)
14/04/26	Log out	Black box testing, Functionality testing, Security testing, Session management testing.	No test data required, the log in system must be fully functional and allow users to create accounts and save their data to a database and the log out button should be clear and easily visible.
14/04/26	Remember me check box on login page	Black box testing, Functionality testing, Security testing, Session management testing.	No test data required. The user must log into the system and tick the remember me box, close the system then reopen the system. The user should still be logged into the system.
14/04/26	Forgot password button on Log in page	Black box testing, Functionality testing, Security testing, Session management testing.	Test data: Normal: Test@Testemail.com Erroneous: Blank email field Extreme: Email that hasn’t been used to make an account on the system
14/04/26	Link buttons on navigation bar	Black box, Functionality testing, integration testing	No test data required. All navigation bar links should be clicked and tested to be fully functional, the links must redirect the users to the corresponding pages and their control logics must be working flawlessly.
16/04/26	Logo on navigation bar	Black box, Functionality testing, integration testing	No test data needed. Homepage should be made and fully functional Navbar should be made and fully functional On all pages, clicking the logo on the navigation bar should reload the homepage for easy navigation.
16/04/26	Links on the footer	Black box, Functionality testing, integration testing	No test data required. All pages linked on the footer must be checked to be fully functional. The links on the footer must be fully functional, redirecting the user to all of their corresponding pages, controller logic must be working flawlessly.

16/04/26	Basket button on the navigation bar	Black box, functionality testing, integration testing	No test data required. The basket button on the navigation bar must be tested to be fully working, on interaction should redirect user to the basket page. Controller logic must be inspected to make sure function works properly.
16/04/26	Price of selected items on the bottom of the basket page	Black box, functionality testing, integration testing	Test data: User must add items to basket from products page, user must click basket button on navigation bar to enter basket page, user must then select items to checkout and see if the price updates to match their choices. When an item is ticked for checkout, total price counter should update to reflect the additional price of the product that has been added. Controller logic must be inspected that this feature has been implemented efficiently and flawlessly.
16/04/26	Checkout using the checkout button on the bottom of the basket page	Black box, functionality testing, integration testing	Test data: User must add items to basket from products page, user must click basket button on navigation bar to enter basket page, user must then select items to checkout, User must then click the checkout button and fully checkout. The user must not run into any issues when checking out their products to create an order, successful checkout should reflect in the dashboard of the admins, producers and selected user all with correct parameters. Controller logic must be checked to make sure feature has been implemented efficiently and correctly.
16/04/26	Accessibility features	Black box Functionality Integration	Accessibility features must be fully functional and easily accessible by users. Tester should test each accessibility feature to see if it

			works. For example, dark mode, high contrast mode, text resize and text-to-speech. No test data needed.
20/04/26	Adding products	Black box, white box, functionality testing, integration testing, security testing	Test data: Normal: ProductName: “Fresh Tomatoes”, Price:2.99, Stock: 50, Description: “Locally grown”, IsAvailable: true. Erroneous: Price: -1, stock: -5, ProductName: “” Extreme: Price: 0.01, stock:0 Tester must be logged in using an admin or producer account.
20/04/26	Updating products stock/quantity	Black Box, Functionality testing, integration testing	Tester must be logged in with an account holding the producer or admin role, at least one product must exist, edit page must be fully implemented. Test data: Normal: Update stock from 50 to 30 Erroneous: Stock to -1 Extreme: Stock to 0
20/04/26	Filtering products	Black Box, functionality testing, integration testing	Store page must be fully implemented, multiple products with different categories, a range of names and prices must exist in the database Test data: Filter by stamp: set stamp filter to “organic” Filter by price ranges. Normal price range: £10 - £20 Erroneous price range: £-12 -£10 Filter by producer: Normal: A valid producer Erroneous: Blank field
20/04/26	Discount code at checkout	Black box, Functionality testing, security testing, integration testing	Checkout page must be fully implemented, at least 1 discount code must exist in the database Test Data: Normal: Valid active code “SUMMER10” Erroneous: Invalid code “FAKECODE”

			Extreme: Deactivated code “WINTER25” (a code that has its status as deactivated)
20/04/26	Delivery vs pickup selection at checkout	Black Box, functionality testing, integration testing	Checkout page and logic must be fully implemented, IsDelivery field must be functional. Test data: Normal: Select Delivery, enter valid address. (Delivery fee should be added to total) Erroneous: Select delivery but leave address empty (should not allow to proceed to checkout)
20/04/26	Stamp management	Black box, Functionality testing, security testing, integration testing	Tester must be logged into the system with an admin account. Stamps page must be fully implemented Test Data: Normal: Add Stamp “Organic” with a description. Assign it to a product. Erroneous: Add stamp with empty name. Security: A non admin user should not be able to access stamp management pages at all, test by logging in as a customer and trying the navigation URL link. Access should be blocked.
21/04/26	Order History	Black box, Functionality testing, integration testing	Customer must be logged in and have at least one completed order. Test data: Order history should display all past orders with correct products, quantities, total price and order date. Clicking an order should show full detail. A customer should not be able to view another customers order history.
21/04/26	Duplicate account registration	Black box, functionality testing, security testing, integration testing	Registration page must be fully implemented. An account must already exist in the database with the email Test@Testemail.com. Tester should attempt to register a second account using that same email. The system should disallow this attempt and display the appropriate error message. The

			database should contain only one account with that email.
21/04/26	Unauthorised route access	Black box, functionality testing, security testing, integration testing	A logged-out session must be used. Tester must directly navigate via URL to /Dashboard, /Basket, /Admin and /Producer, each route should redirect the user to the login page rather than rendering the targeted page. Tests with logged in session of a customer account should block /Admin and pages as they do not have sufficient permission.
21/04/26	Producer Login and Dashboard Access	Black box, functionality testing, security testing, integration testing	At least 2 producer account with products must exist and be approved on the database. Tester should login with the producer account and verify the producer dashboard loads correctly. Tester should also test that the producer can only see their own products and cannot view or edit products belonging to other producers
21/04/26	Producer cannot access admin pages	Black box, security testing, integration testing	A producer account must be logged in. Tester must directly navigate using URL to the stamp management page, user management page, and discount code management page. All three should block access and redirect the producer away. The producer role must not have the same permissions as an admin role. No test data required beyond the producer account credentials.
21/04/26	Admin granting stamps to products	Black box, functionality testing, security testing, integration testing	Admin must be logged in. At least one product and at least one stamp must be present in the database. Normal: Admin assigning stamp “organic” to tomatoes Stamp should appear on the products page Erroneous: Admin assigning a stamp to a product that doesn’t exist Should fail and give an error, not

			crash system. Log in as a customer and try to access stamp assignment URL directly: access should be blocked
22/04/26	Admin granting stamps to producers	Black box, functionality testing, security testing, integration testing	Admin must be logged in. At least one producer and at least one stamp must be present in the database. Normal: Admin assigns stamp “Free Range” to “Happy farms”. Stamp should appear on the producer’s profile and producers page Erroneous: Admin assigning a stamp to a producer that doesn’t exist Should fail and give an error, not crash system.
22/04/26	Discount code creation	Black box, functionality testing, security testing, integration testing	Tester must be logged in as an admin, Discount codes page must be fully implemented. Normal: Code: SPRING15 with 15% off, and IsActive set to true. The code should be created successfully and be usable at checkout. Erroneous: Admin attempts to create a code with a blank name. should be rejected and an error message should be thrown. Erroneous: Admin tries to create a code with a percentage value above 100 or below 0. Should be rejected and an error message should be sent.
22/04/26	Adding items to basket	Black box, functionality testing, integration testing	Tester must be logged in as a customer. At least one product with IsAvailable:true and stock greater than 0 must exist. Basket system must be fully implemented. one product with IsAvailable:False should be present. Normal: Tester clicks “Add to basket” on a product. The item should appear in the basket page

			Tester should navigate to the basket page and confirm that it is the correct product, and all of its details match correctly (producer name, stamps, etc.) Erroneous: tester should attempt to add a product that has its IsAvailable set to false, the add to basket button should be disabled.
22/04/26	Basket Persists Across Sessions	Black box, functionality testing, integration testing	Tester should be logged in as a customer. Two products must be in the basket. Tester should log out of account, tester must log back in using the same account, navigate to the basket page. All previously added items should still be present in the basket in their correct quantities. Test Data: customer account credentials, “Organic mixed vegetables (Qty:2)” and “Raw Local Honey” (Qty:1) before logging out. After logging back in both items should appear in the basket unchanged.
22/04/26	Out of stock products	Black box, functionality testing, integration testing	A product must be entered with stock: 0 and IsAvailable: False Normal: Tester should navigate to the store. The product should be marked as unavailable clearly, with a disabled add to basket button A product that was in a customers basket before it went out of stock. When the customer tries to check he product out a warning should be sent and the checkout should not proceed. Controller logic must validate stock levels server-side at checkout
22/04/26	Points Calculated and Granted Accurately	Black box, functionality testing, integration testing	A customer account must exist with a points balance of 0. The points calculation logic must be fully implemented in the controller. Normal: Customer completes an order with a total of £14.50, the

			dashboard should update to show 145 points added to their balance. Normal: Tester completes a second order worth £8.00; an additional 80 points should be added to the already existing ones bringing the total to 225. Erroneous: An order that is cancelled or not completed should not grant any points. Extreme: An order of £0.50 should grant 5 points, confirming calculation and assigning is flawless.
23/04/26	Responsive Design on all digital platforms	Black Box Functionality Integration	All pages must be fully built. CSS responsive breakpoints must be implemented. Tester must resize the browser window to tablet (768px) and mobile (375px) widths and verify: (1) navigation collapses to a hamburger menu or mobile layout, (2) product grids reflow to appropriate column counts, (3) cards maintain 30px radius and drop shadow, (4) text remains readable, (5) buttons are touch-accessible (minimum 44×44px tap target). No test data needed.
23/04/26	Cookies Consent popup	Black Box Functionality Integration	Cookies UI must be built and functional. Cookie consent must be triggered by JavaScript on first visit. Tester must clear cookies and open the site as a new visitor. Tester must verify: (1) the cookie banner appears, (2) clicking Accept All dismisses the banner and stores consent, (3) clicking Decline All dismisses the banner and stores the refusal, (4) the banner does not reappear on subsequent visits until cookies are cleared. No test data needed.

23/04/26	About us page	Black Box Functionality Integration	About Us page must be built and functional. Navigation link to About Us must be implemented. Tester must navigate to About Us and verify the hero section, story text, mission text and values cards are all displayed correctly. Values cards must have 30px border-radius and drop shadow. Page must render in both modes. No test data needed.
23/04/26	Privacy policy page	Black Box Functionality Integration	Privacy Policy page must be built and functional. Footer link to Privacy Policy must be implemented. Tester must navigate to the Privacy Policy page and verify all data usage and protection information is displayed. Page must render correctly in both light and dark mode. No test data needed.
23/04/26	Dashboard Orders (Update order status)	White Box Black Box Functionality Security Integration	Dashboard Orders tab must be built and functional. Orders must exist in the database for the logged-in Producer's products. Database migrations must be applied. Tester must navigate to the Orders tab, click Mark Shipped on a Processing order and verify the status badge updates to Shipped. Tester must then click Mark Delivered and verify the badge updates to Delivered. Completed orders must show no action button. No test data needed.
23/04/26	Dashboard (Remove product)	White Box Black Box Functionality Security Integration	Dashboard Stock tab must be built and functional. At least one product must exist for the logged-in Producer. Tester must click the delete button on a product row and verify: (1) a confirmation prompt is shown, (2)

			confirming removes the product from the stock table and the Products page, (3) cancelling the prompt leaves the product unchanged. No test data needed.
23/04/26	Dashboard (Add new product)	White Box Black Box Functionality Security Integration	Dashboard Stock tab must be built and functional. Add Product modal must be implemented. Database migrations must be applied. Tester must click the Add Product button, fill in the modal form and submit. Tester must verify the new product appears in the stock table and on the Products page. Tester must also submit with invalid/missing data and verify appropriate validation errors are shown. **Test Data:** Normal Name: Organic Courgettes Category: Fruits & Veg Price: £1.79 Stock: 40 Erroneous Name: (empty) Price: -5 Stock: -10
24/04/26	Dashboard (Recent orders Table)	White Box Black Box Functionality Integration	Dashboard must be built and functional. Orders must exist in the database for the logged-in Producer's products. Database migrations must be applied. Tester must verify the Recent Orders table shows the correct order ID, customer name (anonymised), items, total and status badge. Clicking View All must route to the full Orders sub-tab.

			No test data needed.
24/04/26	Dashboard (Stock management. View/edit stock)	White Box Black Box Functionality Security Integration	Dashboard Stock tab must be built and functional. Products must exist in the database for the logged-in Producer. Database migrations must be applied. Tester must navigate to the Stock tab and verify: (1) all the Producer's products are listed, (2) status badges correctly reflect In Stock (>20), Low Stock (<20) and Out of Stock (0), (3) editing the stock input and clicking Save updates the value in the database and refreshes the badge. Test data: Normal — set stock to 50: badge must show In Stock Erroneous — set stock to -5: must show validation error Extreme — set stock to 0: badge must show Out of Stock and product hidden from storefront

Visual Design

https://www.figma.com/design/1EqeVTI5KxbxTRcD0BxGgB/GreenField-Local-Hub-Visual-Design-And-Annotations?node-id=0`` ``-1&t=kx2QjfxyFKHHoj72-1
(If this link doesn't work, please get in contact with my college)

Styles

Font Family = Inter, SF Pro Display, sans serif, space grotesk

Dark Mode

Background

- Body: #0B0F14 (Deep blue-black, avoids pure black to help with eye comfort)
- Foreground Elements: #111827 (Cards / Surfaces, for slight elevation contrast)

Font

- Normal: #E5E7EB
- Lighter: #9CA3AF

Interactive

- Normal: #22C55E
- Hover/Selected: #4ADE80

Light Mode

Background

- Body: #F7FDF9 (Very light green-tinted white)
- Foreground Elements: #FFFFFF

Font

- Normal: #111827 For strong readability
- Lighter: #6B7280

Interactive

- Normal: #16A34A
- Hover/Selected: #15803D

Buttons/Cards

- Corners = 12px. Modern, soft.
- Background =
  - Dark: #111827
    Light: #FFFFFF
- No outline
- Shadow =
  - Light: Soft lift (0 4px 12px rgba(0,0,0,0.05))
  - Dark: subtle glow (0 0 0 1px rgba(34,197,94,0.15))

Accent Colours

- Discounts/loyalty offers: #84CC16 Energetic Lime Green
- Success messages: #22C55E
- Warnings: #F59E0B
- Errors: #EF4444 abilit